

**Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Херсонский государственный педагогический университет»**

Институт информационных технологий

Кафедра программной инженерии и информационных технологий

ПРАКТИЧЕСКАЯ РАБОТА
по дисциплине:
«Современные операционные системы и базы данных»
на тему:
«Обработка данных с помощью запросов»

Выполнил:
Студент гр.: 12-25РПм
Трифонов Д.С.

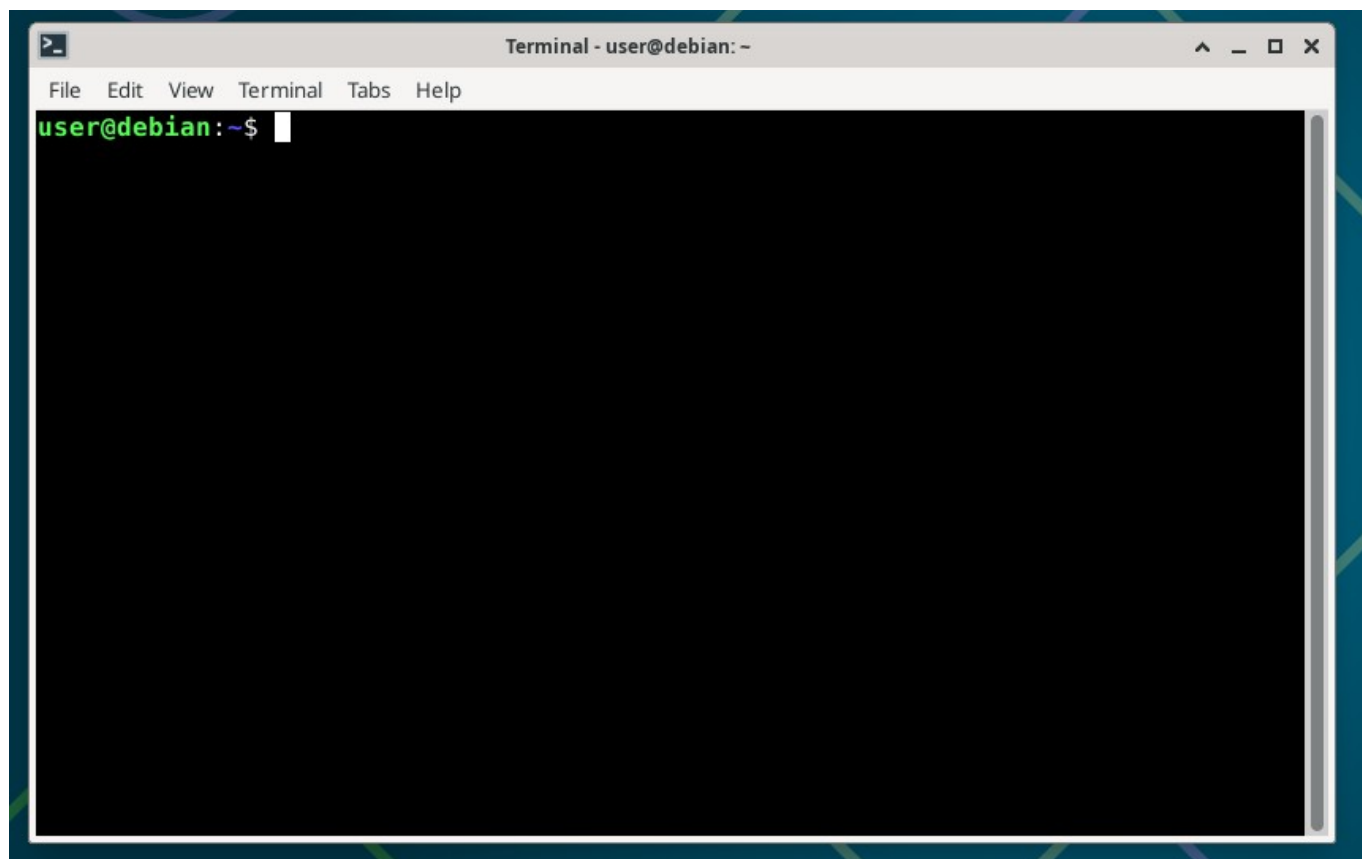
Проверил
Преподаватель:
Алешов В.В.

Оценка: _____
Дата: 25.03.2026

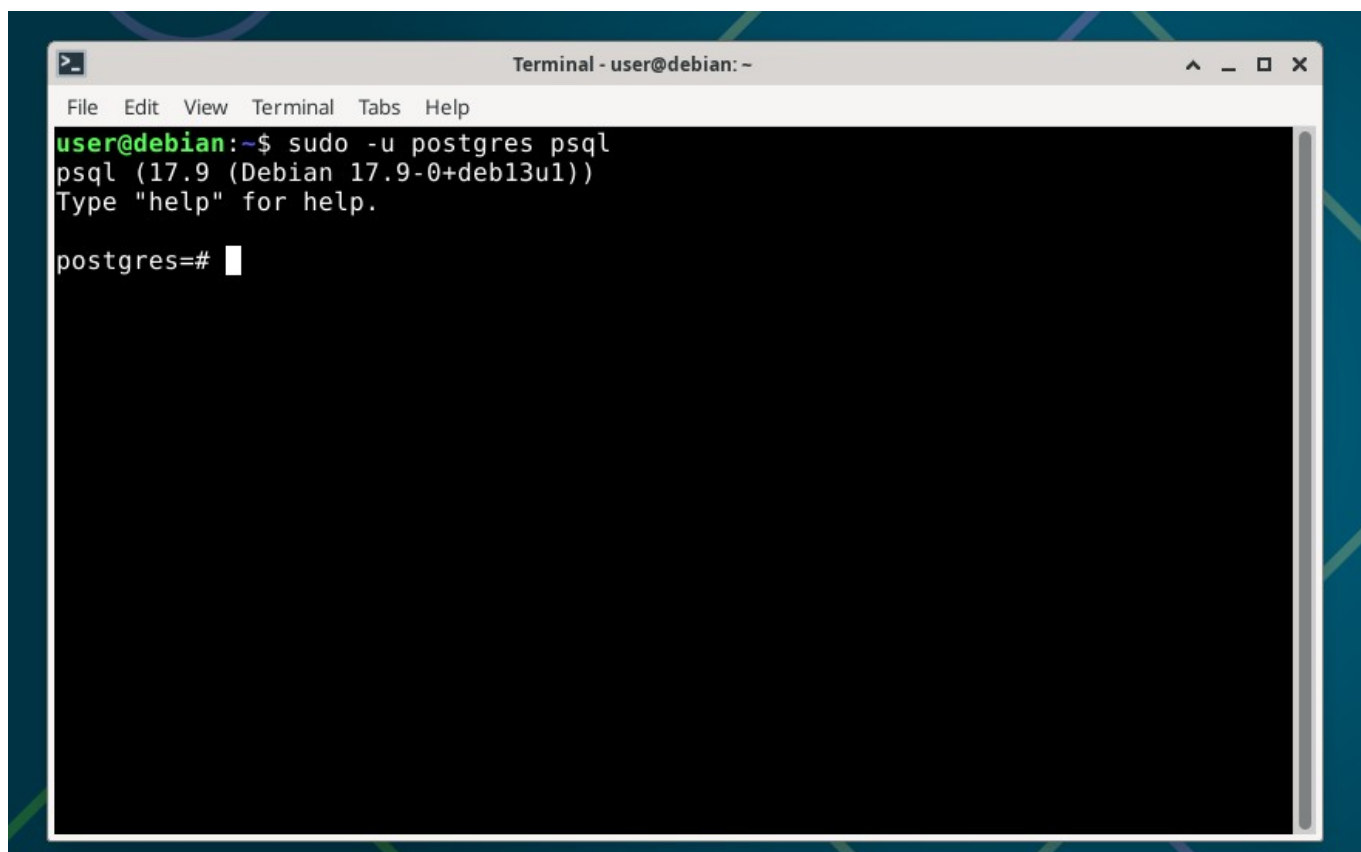
Херсон, 2026

ПРАКТИЧЕСКАЯ РАБОТА

Открываем терминал



Запускаем PostgreSQL

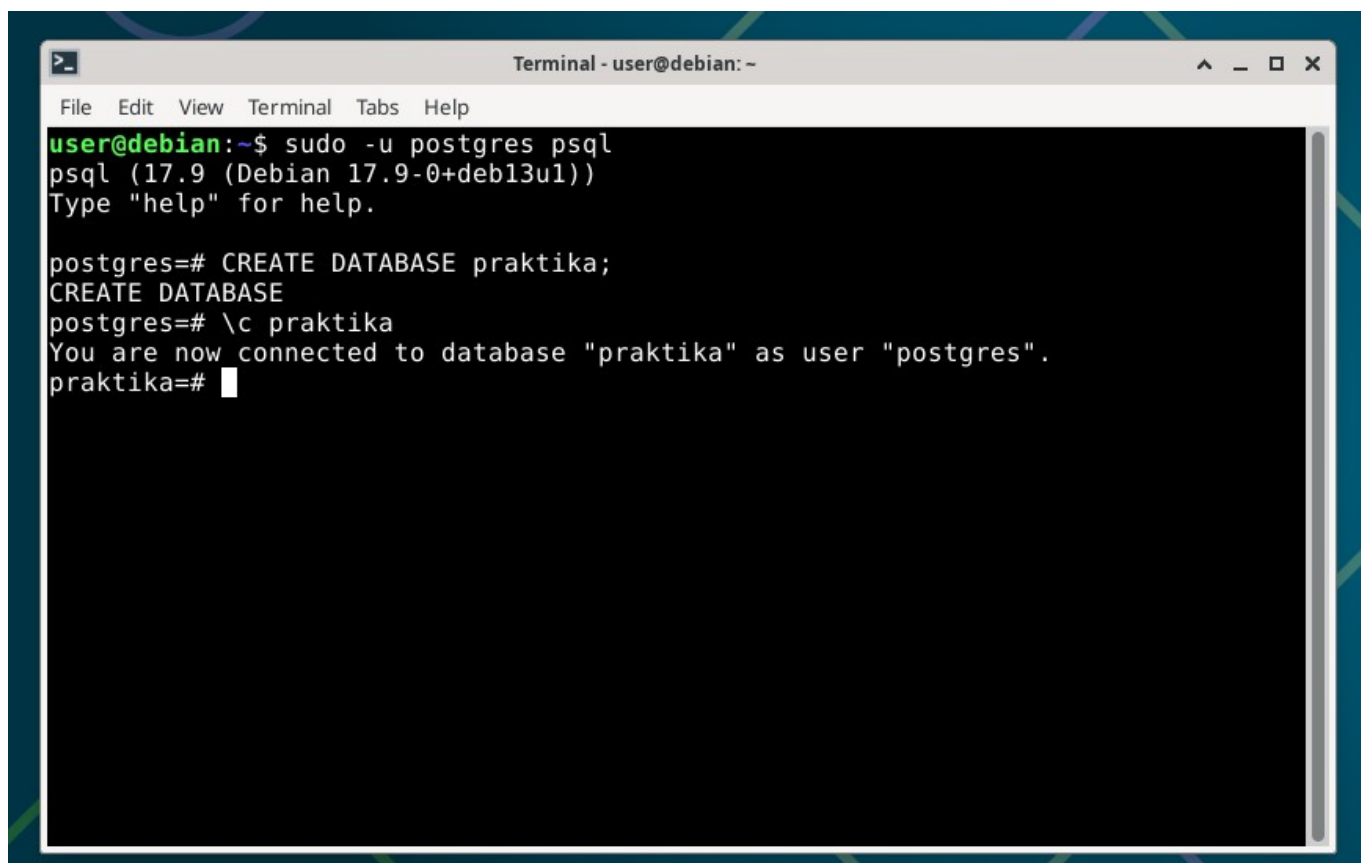


A terminal window titled "Terminal - user@debian: ~" with a menu bar (File, Edit, View, Terminal, Tabs, Help). The prompt is `user@debian:~$`. The user enters `sudo -u postgres psql`. The terminal shows the output: `psql (17.9 (Debian 17.9-0+deb13u1))` and `Type "help" for help.`. The prompt changes to `postgres=#` with a cursor.

```
Terminal - user@debian: ~
File Edit View Terminal Tabs Help
user@debian:~$ sudo -u postgres psql
psql (17.9 (Debian 17.9-0+deb13u1))
Type "help" for help.

postgres=#
```

Создаём базу данных для практики

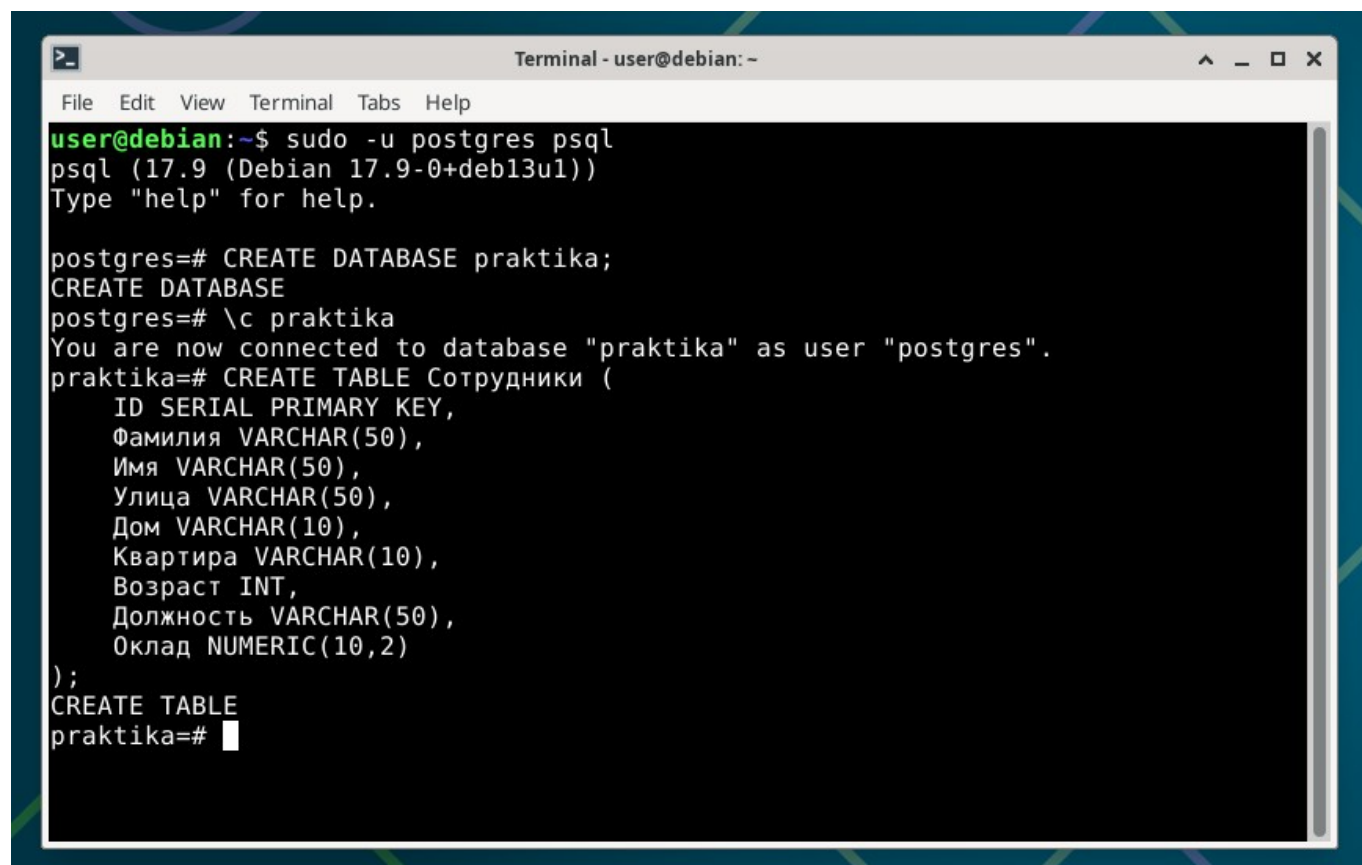


A terminal window titled "Terminal - user@debian: ~" with a menu bar (File, Edit, View, Terminal, Tabs, Help). The prompt is `user@debian:~$`. The user enters `sudo -u postgres psql`. The terminal shows the output: `psql (17.9 (Debian 17.9-0+deb13u1))` and `Type "help" for help.`. The prompt changes to `postgres=#`. The user enters `CREATE DATABASE praktika;`. The terminal shows the output: `CREATE DATABASE`. The prompt changes to `postgres=#`. The user enters `\c praktika`. The terminal shows the output: `You are now connected to database "praktika" as user "postgres".`. The prompt changes to `praktika=#` with a cursor.

```
Terminal - user@debian: ~
File Edit View Terminal Tabs Help
user@debian:~$ sudo -u postgres psql
psql (17.9 (Debian 17.9-0+deb13u1))
Type "help" for help.

postgres=# CREATE DATABASE praktika;
CREATE DATABASE
postgres=# \c praktika
You are now connected to database "praktika" as user "postgres".
praktika=#
```

Создай таблицу Сотрудники с примерными полями

A terminal window titled "Terminal - user@debian: ~" with a menu bar (File, Edit, View, Terminal, Tabs, Help). The terminal shows the following commands and output:

```
user@debian:~$ sudo -u postgres psql
psql (17.9 (Debian 17.9-0+deb13u1))
Type "help" for help.

postgres=# CREATE DATABASE praktika;
CREATE DATABASE
postgres=# \c praktika
You are now connected to database "praktika" as user "postgres".
praktika=# CREATE TABLE Сотрудники (
    ID SERIAL PRIMARY KEY,
    Фамилия VARCHAR(50),
    Имя VARCHAR(50),
    Улица VARCHAR(50),
    Дом VARCHAR(10),
    Квартира VARCHAR(10),
    Возраст INT,
    Должность VARCHAR(50),
    Оклад NUMERIC(10,2)
);
CREATE TABLE
praktika=#
```

Добавляем несколько тестовых записей

```

Terminal - user@debian: ~
File Edit View Terminal Tabs Help
CREATE DATABASE
postgres=# \c praktika
You are now connected to database "praktika" as user "postgres".
praktika=# CREATE TABLE Сотрудники (
    ID SERIAL PRIMARY KEY,
    Фамилия VARCHAR(50),
    Имя VARCHAR(50),
    Улица VARCHAR(50),
    Дом VARCHAR(10),
    Квартира VARCHAR(10),
    Возраст INT,
    Должность VARCHAR(50),
    Оклад NUMERIC(10,2)
);
CREATE TABLE
praktika=# INSERT INTO Сотрудники (Фамилия, Имя, Улица, Дом, Квартира, Возраст,
Должность, Оклад)
VALUES
('Иванов', 'Иван', 'Ленина', '10', '5', 30, 'Менеджер', 50000),
('Петров', 'Пётр', 'Советская', '12', '10', 28, 'Программист', 70000),
('Сидоров', 'Сидор', 'Мира', '5', '2', 45, 'Бухгалтер', 45000),
('Кузнецов', 'Алексей', 'Ленина', '7', '3', 38, 'Менеджер', 55000);
INSERT 0 4
praktika=#

```

Смотрим всех сотрудников

```

Terminal - user@debian: ~
File Edit View Terminal Tabs Help

id | Фамилия | Имя | Улица | Дом | Квартира | Возраст | Должность | Оклад
-----+-----+-----+-----+-----+-----+-----+-----+-----
  1 | Иванов | Иван | Ленина | 10 | 5 | 30 | Менеджер | 50000.00
  2 | Петров | Пётр | Советская | 12 | 10 | 28 | Программист | 70000.00
  3 | Сидоров | Сидор | Мира | 5 | 2 | 45 | Бухгалтер | 45000.00
  4 | Кузнецов | Алексей | Ленина | 7 | 3 | 38 | Менеджер | 55000.00
(4 rows)

(END)

```

Создаем поле «Адрес» через SELECT

```

praktika=# INSERT INTO Сотрудники (Фамилия, Имя, Улица, Дом, Квартира, Возраст,
Должность, Оклад)
VALUES
('Иванов', 'Иван', 'Ленина', '10', '5', 30, 'Менеджер', 50000),
('Петров', 'Пётр', 'Советская', '12', '10', 28, 'Программист', 70000),
('Сидоров', 'Сидор', 'Мира', '5', '2', 45, 'Бухгалтер', 45000),
('Кузнецов', 'Алексей', 'Ленина', '7', '3', 38, 'Менеджер', 55000);
INSERT 0 4
praktika=# SELECT * FROM Сотрудники;
praktika=# SELECT
    Фамилия,
    Имя,
    Улица || ', д.' || Дом || ', кв.' || Квартира AS Адрес
FROM
    Сотрудники;

```

Фамилия	Имя	Адрес
Иванов	Иван	Ленина, д.10, кв.5
Петров	Пётр	Советская, д.12, кв.10
Сидоров	Сидор	Мира, д.5, кв.2
Кузнецов	Алексей	Ленина, д.7, кв.3

```

(4 rows)
praktika=#

```

Отбираем сотрудников по возрасту 25–40 лет

```

praktika=# SELECT * FROM Сотрудники WHERE Возраст >= 25 AND Возраст <= 40;

```

id	Фамилия	Имя	Улица	Дом	Квартира	Возраст	Должность	Оклад
1	Иванов	Иван	Ленина	10	5	30	Менеджер	50000.00
2	Петров	Пётр	Советская	12	10	28	Программист	70000.00
4	Кузнецов	Алексей	Ленина	7	3	38	Менеджер	55000.00

```

(3 rows)
(END)

```

Объединяем оба задания в один запрос

```

Terminal - user@debian: ~
File Edit View Terminal Tabs Help
Кузнецов | Алексей | Ленина, д.7, кв.3
(4 rows)

praktika=# SELECT *
FROM Сотрудники
WHERE Возраст BETWEEN 25 AND 40;
praktika=# SELECT
    Фамилия,
    Имя,
    Улица || ', д.' || Дом || ', кв.' || Квартира AS Адрес,
    Возраст,
    Должность,
    Оклад
FROM
    Сотрудники
WHERE Возраст BETWEEN 25 AND 40;
Фамилия | Имя | Адрес | Возраст | Должность | Оклад
-----+-----+-----+-----+-----+-----
Иванов | Иван | Ленина, д.10, кв.5 | 30 | Менеджер | 50000.00
Петров | Пётр | Советская, д.12, кв.10 | 28 | Программист | 70000.00
Кузнецов | Алексей | Ленина, д.7, кв.3 | 38 | Менеджер | 55000.00
(3 rows)

praktika=#
  
```

Сортировка по фамилии

```

Terminal - user@debian: ~
File Edit View Terminal Tabs Help
Иванов | Иван | Ленина, д.10, кв.5 | 30 | Менеджер | 50000.00
Петров | Пётр | Советская, д.12, кв.10 | 28 | Программист | 70000.00
Кузнецов | Алексей | Ленина, д.7, кв.3 | 38 | Менеджер | 55000.00
(3 rows)

praktika=# SELECT
    Фамилия,
    Имя,
    Улица || ', д.' || Дом || ', кв.' || Квартира AS Адрес,
    Возраст,
    Должность,
    Оклад
FROM
    Сотрудники
WHERE Возраст BETWEEN 25 AND 40
ORDER BY Фамилия ASC;
Фамилия | Имя | Адрес | Возраст | Должность | Оклад
-----+-----+-----+-----+-----+-----
Иванов | Иван | Ленина, д.10, кв.5 | 30 | Менеджер | 50000.00
Кузнецов | Алексей | Ленина, д.7, кв.3 | 38 | Менеджер | 55000.00
Петров | Пётр | Советская, д.12, кв.10 | 28 | Программист | 70000.00
(3 rows)

praktika=#
  
```

Перечень контрольных вопросов

1. Какие операции реляционной алгебры реализуются с помощью запросов?

С помощью SQL-запросов реализуются следующие основные операции реляционной алгебры:

- **Селекция (Selection)** – выборка строк (кортежей) по заданному условию.
Пример: `SELECT * FROM Сотрудники WHERE Возраст > 30;`
- **Проекция (Projection)** – выборка определённых столбцов (атрибутов).
Пример: `SELECT Фамилия, Имя FROM Сотрудники;`
- **Соединение (Join)** – объединение данных из нескольких таблиц на основе ключей.
Пример: `SELECT Товары.Наименование, Категории.Название FROM Товары INNER JOIN Категории ON Товары.Код_категории = Категории.Код;`
- **Агрегация и группировка (Aggregation & Grouping)** – вычисление сумм, среднего, количества и других итоговых значений.
Пример: `SELECT Категория, SUM(Продажи.Сумма) FROM Продажи GROUP BY Категория;`
- **Объединение множеств (Union, Intersect, Except)** – операции над целыми наборами данных.

2. Каковы основные преимущества использования запросов перед непосредственной работой с таблицами?

- **Фильтрация данных** – можно выбирать только нужные строки или столбцы.
 - **Агрегация и вычисления** – легко считать суммы, средние, максимумы и другие показатели.
 - **Соединение таблиц** – объединять данные из разных таблиц без изменения исходных данных.
 - **Переиспользуемость** – один запрос можно использовать многократно в формах, отчётах и скриптах.
 - **Безопасность данных** – можно работать с подмножеством данных, не трогая исходную таблицу.
 - **Автоматизация** – запросы можно запускать программно через скрипты или макросы.
-

3. Что такое результирующий набор записей? Каковы его свойства и способы использования?

- **Результирующий набор записей** (Result Set) – это таблица данных, которая создаётся в результате выполнения SQL-запроса.
 - **Свойства:**
 - Динамический – изменяется при изменении исходных данных.
 - Может содержать любое количество строк и столбцов, в зависимости от запроса.
 - Не обязательно хранится в базе – чаще используется временно для анализа.
 - **Использование:**
 - Для просмотра и анализа данных.
 - В качестве источника данных для форм и отчётов.
 - Для передачи данных в другие приложения или скрипты.
-

4. Как осуществляется сортировка данных в запросе?

- Сортировка выполняется с помощью оператора **ORDER BY**.
- Можно указать один или несколько столбцов и направление сортировки:
 - **ASC** – по возрастанию (по умолчанию).
 - **DESC** – по убыванию.
- Пример:

```
SELECT Фамилия, Имя, Оклад  
FROM Сотрудники  
ORDER BY Оклад DESC, Фамилия ASC;
```

- В этом примере сотрудники сначала сортируются по зарплате по убыванию, затем по фамилии по возрастанию.
-

5. Что такое группировка данных? Для чего используется оператор **GROUP BY**?

- **Группировка данных** – объединение строк с одинаковыми значениями одного или нескольких столбцов для выполнения агрегатных вычислений.

- **Оператор GROUP BY** используется для формирования групп перед применением агрегатных функций: SUM(), COUNT(), AVG(), MAX(), MIN().

- Пример:

```
SELECT Должность, COUNT(*) AS Количество_сотрудников, AVG(Оклад)
AS Средний_оклад
FROM Сотрудники
GROUP BY Должность;
```

- Результат: таблица с количеством сотрудников и средним окладом для каждой должности.

6. В чём различие между условиями WHERE и HAVING? Когда применяется каждое из них?

- **WHERE** – фильтрует строки **до группировки**, работает с отдельными записями.
- **HAVING** – фильтрует группы **после применения GROUP BY**, работает с агрегатными значениями.

- Пример:

-- Сначала фильтруем отдельных сотрудников по возрасту

```
SELECT Должность, COUNT(*) AS Количество
FROM Сотрудники
WHERE Возраст >= 25
GROUP BY Должность
```

-- Потом фильтруем группы по количеству сотрудников

```
HAVING COUNT(*) > 2;
```

- Идея: **WHERE** – для отдельных строк, **HAVING** – для групп.